

PROGETTO MOBILE PROGRAMMING

A.A 2025-2026

STUDY PLANNER & EXAM TRACKER APP

Pianificazione e monitoraggio della carriera accademica

Componenti del Gruppo:

Angelo Palladino - Matr. 0612709336

Chiara Stabile - Matr. 0612709345

Pasquale Santoriello - Matr. 0612710199

Roberto La Brocca - Matr. 0612709464

Michele Tamburro - Matr. 0612709882

1. Descrizione dell'applicazione

1.1 Obiettivo dell'applicazione

L'applicazione Study Planner & Exam Tracker è stata concepita come uno strumento di supporto personale per uno studente universitario, il suo obiettivo primario è infatti centralizzare e semplificare la gestione della carriera accademica tramite un ecosistema mobile che permetta la pianificazione temporale dello studio, il monitoraggio di esami e medie, il cronometraggio in tempo reale delle sessioni di studio, e la gestione di un'agenda giornaliera, offrendo in questo modo un quadro analitico delle performance dello studente.

In particolare tramite il suo utilizzo uno studente risulta essere in grado di monitorare l'andamento dei corsi attivi nel semestre, tenere traccia degli esami passati, futuri e programmati, pianificare le ore di studio giornaliere e confrontarle con il tempo effettivo dedicato all'apprendimento, e visualizzare un cruscotto di controllo per valutare tempestivamente lo stato di avanzamento ed eventualmente ottimizzare la gestione del tempo.

1.2 Tipologia di utenti a cui è rivolta

Il destinatario finale di questa applicazione è lo studente universitario di qualsiasi facoltà, anno di iscrizione o ordinamento, grazie alle sue molteplici funzionalità, infatti, l'app è in grado di adattarsi a profili differenti: dallo studente al primo anno che necessita di comprendere il ritmo e il metodo di studio, fino al laureando che ha invece l'esigenza di calcolare proiezioni precise sui voti ed organizzare le scadenze delle tesi o delle ultime sessioni.

1.3 Problema che l'app intende risolvere

L'efficienza dell'app è determinata dal fatto che un'organizzazione universitaria impone allo studente una gestione autonoma del proprio tempo e un carico di esami e attività notevole, per cui esso si ritrova spesso costretto ad utilizzare diverse applicazioni di supporto alla gestione, portando ad una frammentazione degli strumenti utilizzati (calendari cartacei per le lezioni, fogli di calcolo per le medie, sveglie/timer generici per lo studio).

L'applicazione Study Planner & Exam Tracker risulta essere la soluzione perfetta a questa frammentazione grazie alla sua capacità di fondere in un'unica app tali funzionalità, permettendo di associare attività ed esami ad uno specifico corso accademico, raccogliere i dati delle sessioni reali (collegate ad un timer), evidenziando visivamente gli scostamenti per un miglioramento della pianificazione futura, e calcolare la media o aggiornare gli stati in maniera immediata.

1.4 Principali scenari d'uso

L'app si presta a diversi scenari d'uso:

- Scenario A - Gestione Didattica e Storicizzazione della Carriera: lo studente registra un nuovo corso nel proprio piano di studi (ad esempio, Computer Vision da 9 CFU) inserendo la data di inizio e fine del ciclo di lezioni. Di seguito, pianifica l'appello d'esame corrispondente. Una volta sostenuto l'esame, in seguito l'utente sfrutta la procedura di verbalizzazione integrata per archiviare il voto: il sistema quindi aggiorna istantaneamente lo stato dell'insegnamento a "Completato" e ricalcola i CFU totali e la media ponderata nella dashboard;
- Scenario B - Pianificazione Granulare Giornaliera: lo studente consulta l'agenda all'interno della schermata di pianificazione per visionare le attività o sessioni del giorno selezionato sul

un calendario mensile. Per focalizzare l'attenzione sulle attività critiche a ridosso delle scadenze, interagisce con i filtri di ricerca presenti nella schermata di pianificazione al di sopra dell'agenda, isolando quindi le attività aventi priorità Alta e associate ad un singolo insegnamento, nascondendo i blocchi temporali stabili delle sessioni oppure le attività marcate come già completate;

- Scenario C - Studio Assistito da Timer ed Elaborazione delle Statistiche: lo studente intende avviare una sessione di ripasso mirata; pertanto accede alla schermata del Timer, seleziona il macro-blocco di studio e l'attività atomica su cui lavorare, avviando il countdown nativo. Se l'utente decide di passare ad un altro compito, il timer intercetta il cambio a runtime memorizzando la sessione appena conclusa nell'archivio storico giornaliero: questo flusso alimenta in maniera automatica i grafici comparativi settimanali presenti nella dashboard generale.
- Scenario D - Interazione Rapida Globale (Quick Add): durante la visualizzazione del grafico sull'andamento delle medie nella Dashboard, lo studente ricorda una scadenza urgente per un progetto universitario appena annunciata. Invece di interrompere la sua sessione per navigare manualmente alla sezione Planner, seleziona il pulsante fluttuante centrale (+) integrato nella Tab Bar: all'apertura del menu, seleziona "Pianifica Attività/Sessione", compila i dettagli del task e, alla chiusura, riprende la visualizzazione dei grafici esattamente dal punto in cui si trovava.
- Scenario E - Filtraggio selettivo e ricerca rapida: con l'accumularsi degli insegnamenti registrati nella sezione Carriera, lo studente ha bisogno di recuperare rapidamente i dati di un corso specifico ancora attivo. Accedendo alla schermata Carriera, utilizza la barra di ricerca superiore digitando il nome del docente, inoltre per escludere i corsi già superati, seleziona in parallelo lo stato "In corso", a questo punto il catalogo si riduce mostrando la sola scheda di interesse.
- Scenario F - Analisi delle Performance e Confronto delle Medie: al fine di fare una proiezione sulla media finale dopo aver superato un esame, lo studente naviga nella Dashboard ed interagisce con il toggle "Tempo / Media", a questo punto l'interfaccia scambia l'istogramma delle ore con un grafico ad andamento lineare in cui lo studente confronta la curva della media aritmetica con quella della media ponderata (pesata sui CFU dei corsi superati) valutando graficamente l'impatto sulla propria carriera.

2. Requisiti

2.1 Tabella dei Requisiti Funzionali e Feature Implementate

ID	Area Funzionale	Requisito dettagliato	Stato
----	-----------------	-----------------------	-------

RF-01	Cruscotto (Dashboard)	Grafico delle ore di studio settimanali: istogramma a barre verticali che confronta le ore effettive registrate (timer) e pianificate (agenda).	Implementato
RF-02	Cruscotto (Dashboard)	Grafico delle medie: istogramma temporale con un grafico ad andamento lineare delle medie.	Implementato
RF-03	Cruscotto (Dashboard)	Grafico a ciambella (Donut Chart) sul progresso esami: suddivisione visiva tra esami Superati, Programmati e Da Iniziare.	Implementato
RF-04	Cruscotto (Dashboard)	Visualizzazione di: CFU Guadagnati (es. 0 / 54), Media Ponderata e numero di Attività ancora da completare.	Implementato
RF-05	Cruscotto (Dashboard)	Timeline dei Corsi Attivi: monitoraggio della progressione di ciascun corso basato sulla data corrente rispetto alle date di inizio e fine corso.	Implementato
RF-06	Pianificazione (Planner)	Calendario Mensile Interattivo: calendario a griglia per selezionare i giorni dell'anno e visualizzare l'agenda specifica del giorno scelto.	Implementato
RF-07	Pianificazione (Planner)	Filtraggio: possibilità di filtrare per specifico corso, per stato (Tutte/Da	Implementato

		completare/Complete) e per priorità (Tutte/Alta/Media/Bassa).	
RF-08	Pianificazione (Planner)	Gestione e visualizzazione attività e sessioni: presenza di card dettagliate con titolo, corso, tempo stimato, priorità (indicata dal pallino colorato), checkbox di completamento e opzioni Edit/Delete.	Implementato
RF-09	Navigazione	Pulsante fluttuante centralizzato (+) (<i>Quick Add</i>): bottone posizionato nella tab bar inferiore che consente l'inserimento rapido di corsi, esami, sessioni o attività da qualsiasi sezione dell'app.	Implementato
RF-10	Carriera (Academic)	Visualizzazione di: elenco dei corsi didattici ed elenco degli appelli d'esame tramite uno switch nell'header.	Implementato
RF-11	Carriera (Academic)	Barra di ricerca: filtro per nome del corso o cognome del docente.	Implementato
RF-12	Carriera (Academic)	Filtraggio: filtri sullo stato (Tutti/Da iniziare/In corso/Completato") per monitorare l'avanzamento di corsi e esami.	Implementato
RF-13	Carriera (Academic)	Schede corso: card dettagliate contenenti nome, docente,	Implementato

		semestre, CFU, stato e opzioni Edit/Delete di corsi e esami.	
RF-14	Carriera (Academic)	Dettaglio esami e corsi: possibilità di cliccare su un corso o esame per esaminare in dettaglio tutte le relative informazioni memorizzate.	Implementato
RF-15	Timer	Display Timer: cronometro integrato che permette allo studente di tracciare in tempo reale la durata dei singoli task.	Implementato
RF-16	Timer	Gestione: pulsanti "STOP" e "AVVIA" per gestire gli avvii e le interruzioni.	Implementato
RF-17	Timer	Storico Giornaliero: elenco dei task conclusi con orario, durata effettiva ed esito.	Implementato

2.2 Feature considerate ma non implementate

Sono state prese in esame ulteriori funzionalità dell'app, decidendo di non implementarle per motivazioni di ottimizzazione dell'architettura:

- **Caricamento di file allegati pesanti (PDF delle lezioni):** escluso poiché la serializzazione di file binari pesanti avrebbe rallentato sensibilmente le transizioni di navigazione.
- **Suggerimento automatico di attività in base agli esami imminenti:** esclusa per non delegare la pianificazione ad algoritmi che avrebbero ridotto il controllo manuale dell'agenda da parte dello studente, preferendo invece implementare come feature avanzate il calendario e il timer.

2.3 Feature avanzate scelte

Le seguenti funzionalità non erano necessarie secondo quanto richiesto dalla traccia, ma sono state implementate per migliorare l'esperienza utente (UX):

- **RF-06 - Calendario mensile interattivo:**

Il calendario presente nella nostra applicazione non si limita a mostrare le date, ma funge da centro di controllo dell'agenda, permette di muoversi fluidamente tra i mesi dell'anno, selezionare un giorno specifico e visualizzare all'istante le attività e le sessioni previste per quel giorno. In questo modo lo studente ha un quadro dettagliato dei suoi impegni giornalieri.

- **RF-15, RF-16, RF-17 - Timer integrato:**

L'applicazione integra un cronometro anche si aggancia alle attività e sessioni tramite scorciatoie ad inserimento rapido, esso inoltre non traccia solo il tempo passivamente, ma categorizza gli esiti delle sessioni offrendo un riscontro del livello di focus dello studente.

- **RF-17 - Pulsante Quick Add centrale (+):**

Si tratta di una funzionalità avanzata in quanto Invece di costringere l'utente ad andare nella sezione dedicata per aggiungere un task, il bottone è sempre accessibile al centro della Tab Bar, velocizzando l'inserimento dati da qualsiasi schermata dell'app.

2.4 Limitazioni note

- **Persistenza solo locale (AsyncStorage):**

Per come l'applicazione è stata progettata, se l'utente disinstalla l'app o cambia telefono, i dati vanno persi, tuttavia la traccia esclude esplicitamente lo sviluppo di un backend remoto, per cui per i requisiti di tale applicazione Async Storage garantisce la massima velocità di lettura/scrittura (I/O) senza appesantire l'architettura.

- **Mancanza di file allegati per i materiali dei corsi:**

L'applicazione non permette di caricare PDF reali o slide all'interno del corso, si è preferito convertire questo punto in un campo "Descrizione/Note" testuale strutturato, preservando la leggerezza e la portabilità.

- **Gestione delle date e calendario semplificata**

Il calendario non rileva conflitti di orario reali (es. due lezioni sovrapposte nello stesso momento), in quanto l'app si concentra sulla pianificazione giornaliera e sul rispetto delle scadenze di consegna, che è l'obiettivo didattico del progetto.

3. Progettazione dell'applicazione

3.1 Struttura Generale

L'applicazione è stata sviluppata utilizzando React Native (con Expo) e segue un'architettura modulare basata su componenti. La base di codice è stata strutturata con un approccio logico e gerarchico, mirato a garantire la manutenibilità, la scalabilità e una netta separazione tra la logica di business, la gestione dei dati e l'interfaccia utente (UI).

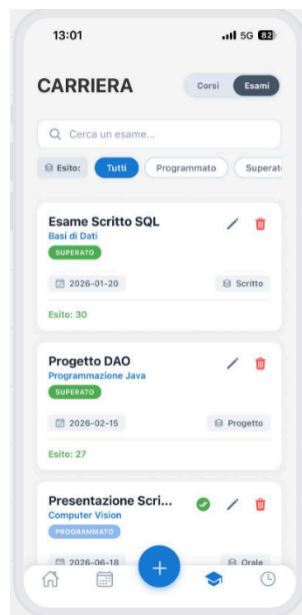
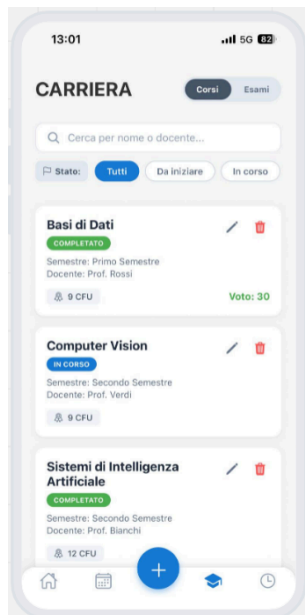
3.2 Schermate principali

L'interfaccia utente è suddivisa in aree funzionali distinte per massimizzare la chiarezza e l'usabilità:

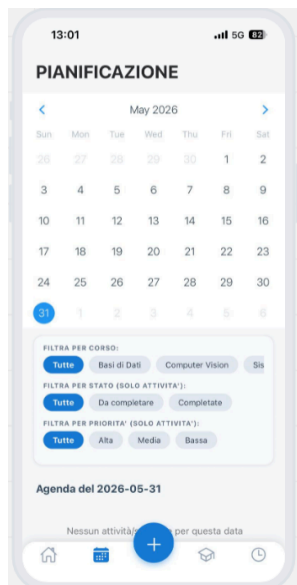
DashboardScreen: Funge da cruscotto riassuntivo. Utilizza grafici avanzati (a torta, a barre e a linee) per fornire all'utente metriche visive immediate sui CFU guadagnati, l'andamento della media voti e il bilancio tra il carico di studio pianificato e quello effettivamente svolto.



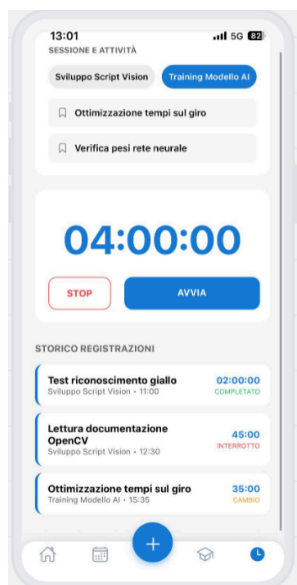
AcademicScreen (Carriera): È l'hub centrale per la gestione del percorso universitario. Permette di visualizzare liste separate per corsi ed esami, offre un sistema di filtraggio incrociato (testuale e per stato) e include l'interfaccia per la verbalizzazione ufficiale degli esiti.



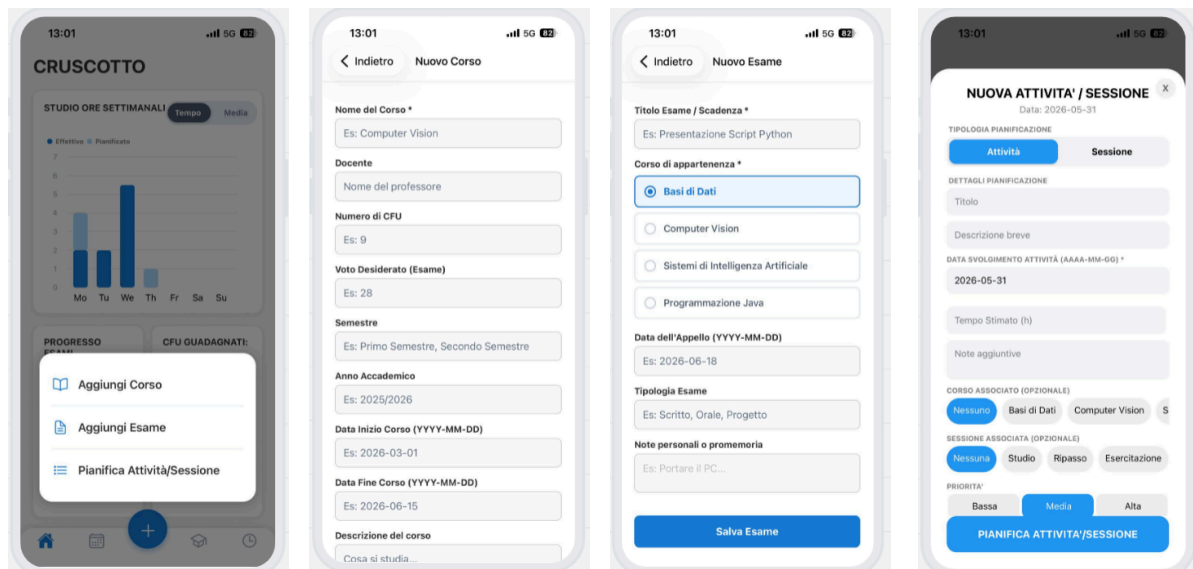
PlanningScreen (Pianificazione): Unisce un calendario interattivo a una to-do list dinamica. Gestisce due tipologie di entità operative: le "Sessioni" (blocchi temporali di studio) e le "Attività" (task specifici e misurabili), consentendo all'utente di filtrarle per priorità, stato di completamento o corso di appartenenza.



TimerScreen: L'interfaccia operativa per il tracciamento del tempo di studio. Si interfaccia dinamicamente con le sessioni pianificate nel Planning e salva in tempo reale uno storico dettagliato delle misurazioni, discriminando tra completamenti naturali e interruzioni manuali.



Schermate di Inserimento (Add Screens): Moduli dedicati all'inserimento e all'aggiornamento dei dati (Corsi ed Esami), dotati di validazione preventiva sui campi (es. controlli di coerenza sulle date di inizio e fine) prima di autorizzare la scrittura in memoria.



3.3 Flusso di navigazione

Il flusso di navigazione utilizza un approccio ibrido, implementato tramite la libreria React Navigation: **Bottom Tab Navigation:** Costituisce lo scheletro dell'applicazione. Permette all'utente uno switch orizzontale rapido tra le quattro macro-aree principali (Dashboard, Carriera, Pianificazione, Timer) mantenendo attivo lo stato di ciascuna schermata in background.

Stack Navigation: Impiegata all'interno dei singoli Tab per gestire la navigazione gerarchica di profondità. Dalla schermata Carriera, ad esempio, l'utente può spingere (push) in primo piano la vista di dettaglio (CourseDetailScreen) o i form di modifica, passando l'oggetto dati selezionato tramite i parametri di rotta (route.params).

Modal Presentation: Per azioni rapide e strettamente contestuali (come la verbalizzazione di un voto o il pop-up di allerta per l'eliminazione di un task), il flusso principale non viene interrotto; vengono invece sovrapposte finestre modali semitrasparenti che catturano il focus dell'utente.

3.4 Organizzazione dei dati

Il sistema di persistenza si basa su AsyncStorage, un database locale di tipo chiave-valore. I dati sono archiviati in formato JSON seguendo un modello relazionale leggero per evitare ridondanze:

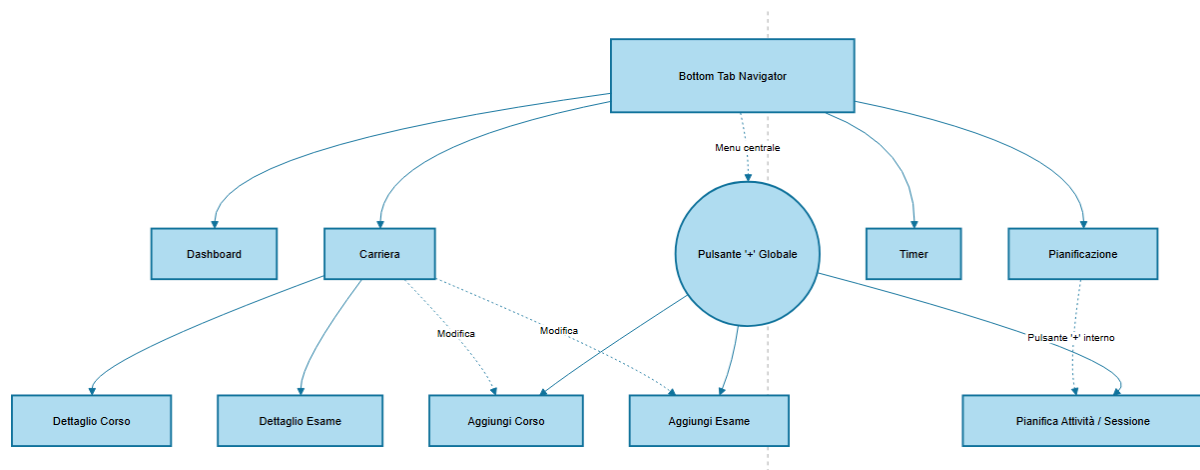
Corsi (@planner_corsi): L'entità genitore principale, contenente anagrafica, crediti formativi e stato di completamento globale.

Esami (@planner_esami): Entità dipendente, legata in modo univoco al proprio corso genitore tramite la proprietà foreign key corso_id.

Attività e Sessioni (@planner_attivita): Per ragioni di efficienza in fase di rendering sul calendario, sono archiviate in un'unica collezione logica. Il codice le differenzia tramite il campo discriminante type (valorizzato a 'sessione' o 'attivita') e ne ricostruisce l'albero gerarchico tramite le chiavi esterne opzionali course_id e session_id.

Storico Timer (@planner_storico_timer): Entità puramente transazionale che archivia i log irreversibili delle sessioni di studio, salvando l'identificativo dell'attività svolta e i minuti effettivamente registrati.

3.5 Diagramma di Navigazione



4. Scelte Tecnologiche

4.1 Framework Utilizzato

L'intera applicazione è stata implementata tramite l'utilizzo di React Native, supportato dall'infrastruttura e il pacchetto di tool di Expo.

Questo Framework ci ha permesso di sviluppare un'applicazione multiplatforma (cross-platform): permette di mantenere una singola base di codice in JavaScript/TypeScript, in grado di essere compilata ed eseguita nativamente sia su sistemi operativi iOS che Android. Questa scelta architetturale ci ha permesso una rapida prototipazione, un'elevata riusabilità dei componenti UI e ottime prestazioni di fluidità (quasi quanto le applicazioni puramente native).

4.2 Librerie Principali

Per realizzare alcuni dei requisiti funzionali, abbiamo dovuto estendere il core dell'applicazione integrando librerie open-source standard del settore:

- **React Navigation** ([@react-navigation/bottom-tabs](#), [@react-navigation/native-stack](#)): Utilizzata per gestire sia la navigazione orizzontale primaria (tramite Tab Bar) sia la navigazione verticale di dettaglio (per i moduli di inserimento) tramite architettura a pila (Stack).
- **React Native Async Storage** ([@react-native-async-storage/async-storage](#)): Libreria utilizzata per l'implementazione del database locale e la persistenza dei dati offline.
- **React Native Calendars** ([react-native-calendars](#)): Modulo specializzato utilizzato nella schermata "Pianificazione" per il rendering di un calendario mensile interattivo, in grado di gestire reattivamente date selezionate e indicatori visivi di priorità.
- **React Native Gifted Charts** ([react-native-gifted-charts](#)): Utilizzata per il rendering dei grafici all'interno del "Cruscotto" (Dashboard), permettono la generazione dinamica di grafici a torta, istogrammi a barre e grafici ad andamento lineare.
- **React Native Modal** ([react-native-modal](#)): Sfruttata per la renderizzazione di grafiche a schermo e form di acquisizione dati (per permettere l'inserimento dei dati da parte dell'utente), si sovrappongono all'interfaccia senza alterare l'albero di navigazione principale.

4.3 Motivazione delle scelte effettuate

La scelta dello stack tecnologico è stata effettuata basandosi sui principi di affidabilità, manutenibilità e user experience. L'utilizzo di React Native ha agevolato e ridotto i tempi di scrittura del codice rispetto all'utilizzo separato di linguaggi puramente nativi come Swift (Apple) e Kotlin (Google).

L'utilizzo di librerie già ottimizzate dalla community (come `gifted-charts` e `calendar`) ci ha permesso di evitare di gestire tecnicismi complicati come il rendering vettoriale, permettendoci di concentrarci sull'implementazione della logica di business e sull'integrità del flusso dei dati.

4.4 Modalità di gestione della persistenza

La persistenza offline dei dati è interamente affidata alla libreria `@react-native-async-storage/async-storage`. Per lo scopo di questa applicazione, un database locale asincrono basato su coppie chiave-valore si è rivelata la scelta architetturale più efficiente in termini di leggerezza e velocità (I/O) rispetto all'inclusione di database relazionali più pesanti (es. SQLite o PostgreSQL).

Per garantire l'atomicità delle transizioni le chiamate al file system sono state centralizzate all'interno di un modulo di servizio dedicato (`storage.js`).

5. Implementazione

5.1 Organizzazione del Codice

Il codice sorgente dell'applicazione segue una rigorosa architettura modulare, concepita per garantire la separazione delle responsabilità (Separation of Concerns), l'isolamento dei flussi logici e la massima manutenibilità del software in un contesto di sviluppo collaborativo. La base di codice è strutturata in quattro directory specializzate all'interno della cartella principale `src`:

- **/components**: questa directory ospita i componenti grafici o molecolari riutilizzabili e i moduli controllati privi di una rotta di navigazione autonoma, ed include il form modale chiamato `AddTaskModal.tsx`;
- **/constants**: questa directory contiene un file di configurazione stilistica globale (file chiamato `Colors.ts`), un file che contiene dei dati fittizi utili ad un test preliminare dell'app (file chiamato `mockData.js`) e lo strato logico di gestione della persistenza sul file system locale (`storage.js`);
- **/navigation**: questa directory contiene il file di configurazione e nidificazione dei navigatori dell'applicazione, chiamato `AppNavigator.tsx`;
- **/screens** e la sotto cartella **/screens/add**: questa directory racchiude i componenti deputati alla resa delle schermate intere registrate nel sistema di navigazione. Le viste sono suddivise tra schermate principali di consultazione (i file `DashboardScreen.js`, `PlanningScreen.tsx`, `AcademicScreen.js`, `TimerScreen.tsx`, `CourseDetailScreen.js`) e form che sono dedicati all'acquisizione valida dei dati (i file `AddCorsoScreen.tsx`, `AddEsameScreen.tsx` e `AddTaskScreen.tsx`).

5.2 Componenti/widget principali

5.2.1 Il Modulo Adattivo delle Attività (AddTaskModal.tsx)

Il componente **AddTaskModal.tsx** rappresenta un modulo di acquisizione e si presenta come un form che permette all'utente di inserire un'attività o sessione e anche di modificare un'attività o sessione precedentemente inserita

L'inizializzazione e il comportamento dinamico del widget sono interamente governati da un hook **useEffect**, ovvero l'hook (cioè una funzione disponibile durante il rendering) di React che serve a gestire i "Side Effects", cioè tutte quelle operazioni che avvengono fuori dal normale flusso di rendering dei componenti; lo **useEffect** in questione monitora le mutazioni della proprietà opzionale **taskToEdit** e dello stato di visibilità **isVisible**:

- Il ruolo di **isVisible**: questa proprietà booleana **controlla l'apertura del modale**. Quando assume il valore **true**, l'hook si attiva per inizializzare i campi; se l'utente chiude il form (e quindi la proprietà **isVisible** assume il valore **false**), il modulo resetta lo stato interno e interrompe il ciclo di rendering del componente per ottimizzare l'allocazione delle risorse;
- **In modalità Modifica**: l'hook rileva l'oggetto **taskToEdit**, estrae i dati storici salvati e popola i rispettivi stati locali (**setTitle**, **setDesc**, **setPriority**, **setNotes**); l'hook esegue inoltre una conversione matematica difensiva sulle metriche temporali: poiché il database memorizza la durata in minuti interi per uniformità di calcolo, il form divide il valore per 60 (**taskToEdit.estimatedTime / 60**) per mostrare all'utente una stringa decimale espressa in ore nei **TextInput**. Infine l'hook applica il metodo **.find()** sull'array dei corsi per effettuare una ricerca inversa e ripristinare visivamente la selezione del nome del corso partendo dall'ID memorizzato
- **In modalità Inserimento**: qualora **taskToEdit** risulti nullo, l'effetto azzera reattivamente tutti i campi del form, predisponendo l'interfaccia a una nuova compilazione pulita.

Per dare forma a questa logica adattiva, l'architettura visiva del file **AddTaskModal.tsx** sfrutta i **componenti nativi fondamentali** di React Native:

- **Modal**: questo componente viene utilizzato come **contenitore principale** per garantire la visualizzazione in sovrimpressione rispetto alla schermata sottostante ed è gestito tramite la proprietà **visible**: questa proprietà nativa è mappata direttamente sullo stato booleano **isVisible** il quale riceve l'input dal componente padre e determina l'effettiva comparsa o scomparsa grafica dell'intero modale;
- **ScrollView**: questo componente è inserito all'interno del modale per racchiudere l'intero form, assicurando la completa usabilità e lo scorrimento dei campi di input anche su dispositivi con schermi ridotti o quando la tastiera virtuale è attiva.

- **TextInput**: questo componente è utilizzato per l'acquisizione dei dati testuali e numerici, configurando dinamicamente la proprietà **keyboardType="numeric"** per i campi temporali in modo da ottimizzare l'esperienza utente.
- **TouchableOpacity**: questo componente è impiegato per la realizzazione dei pulsanti personalizzati di salvataggio (**handleSave**), per i selettori delle priorità e per i filtri a pillola.
- **View e Text**: questi componenti vengono utilizzati rispettivamente per la **strutturazione del layout** (tramite Flexbox) e per la **formattazione dei titoli e delle etichette dei campi**

La flessibilità dell'interfaccia si esprime nel **rendering condizionale inline** agganciato allo stato **type** (che può assumere i valori **'attività'** oppure **'sessione'**):

- se l'utente seleziona la tipologia **Attività**, l'interfaccia utente mostra i selettori di priorità d'urgenza e i campi per le ore stimate ed effettive (quest'ultimo visibile solo in modifica);
- se viene selezionata la tipologia **Sessione**, il form muta a runtime nascondendo la priorità ed introducendo il selettore **durationUnit**, ovvero uno stato che permette di identificare se la durata di una sessione è espressa in ore oppure in giorni: se impostato su "In Ore", espone il campo numerico orario; se impostato su "In Giorni" abilita la visualizzazione di tre TextInput, uno per la Data di Inizio della sessione, uno per la Data di Fine della sessione e un altro per la durata complessiva della sessione in giorni, riconfigurando i vincoli di convalida del form

Al momento del salvataggio, che avviene mediante la function **handleSave**, i dati orari digitati vengono normalizzati arrotondando il valore decimale (**parseFloat**) e moltiplicandolo per 60, registrando nel database esclusivamente minuti interi ed eliminando le anomalie computazionali dei numeri in virgola mobile.

Una volta completata la normalizzazione dei tempi, il modulo inoltra l'oggetto strutturato alla funzione di callback **onSave**. Questo garantisce il perfetto disaccoppiamento del componente: se il form viene aperto all'interno del calendario (**PlanningScreen.tsx**), i dati aggiornano istantaneamente lo stato locale dell'agenda; se invece viene richiamato **tramite il pulsante fluttuante centrale della barra di navigazione**, il controllo passa alla schermata di scelta rapida (**AddSceltaScreen.tsx**), che provvede a salvare direttamente i dati nella memoria del dispositivo.

5.2.2 Il Modulo di Validazione e Stato dei Corsi (AddCorsoScreen.tsx)

Il componente **AddCorsoScreen.tsx** adotta un approccio ingegneristico per garantire la consistenza formale e temporale del piano di studi. Oltre alla precompilazione degli stati locali in modalità di modifica (**corsoDaModificare**), l'elemento cardine del file risiede nelle funzioni di controllo della logica di business:

- **Calcolo Dinamico e Predittivo dello Stato (calcolaStatoCorso)**: il modulo non delega all'utente la scelta dello stato del corso (es. "in corso", "completato", "da iniziare"). Il componente estrae la data odierna di sistema in formato ISO (YYYY-MM-DD) e la confronta matematicamente a runtime con le stringhe inserite nei campi **data_inizio** e **data_fine**, assegnando automaticamente lo stato corretto tramite operatori relazionali.
- **Validazione Cronologica Incrociata (Anno Accademico / Date)**: al click su "Salva", la funzione **handleSalvaCorso** analizza la stringa dell'Anno Accademico (verificando il pattern strutturale AAAA/AAAA tramite lo **splitting** del carattere /). Successivamente, estrae l'anno di competenza dalle date di inizio e fine corso (**substring(0, 4)**) e lancia un alert di errore

bloccante se queste non appartengono al biennio accademico dichiarato. Viene infine applicato un controllo sequenziale per impedire che la data di inizio sia cronologicamente successiva a quella di fine.

- **Ottimizzazione dell'UX e Accessibilità della Tastiera:** per far fronte ai problemi comuni di ostruzione della GUI su dispositivi mobili, il layout è interamente avvolto in un componente **KeyboardAvoidingView** configurato con un **keyboardVerticalOffset** di 90 pixel (attivo su piattaforma iOS) per preservare la visibilità dei campi inferiori. Lo ScrollView sottostante implementa la proprietà **keyboardShouldPersistTaps="handled"** per consentire la chiusura fluida della tastiera al tocco dello sfondo senza compromettere l'interazione sugli elementi web o sui pulsanti di salvataggio.

5.2.3 Il Modulo di Associazione Relazionale degli Esami (AddEsameScreen.tsx)

Il componente **AddEsameScreen.tsx** funge da snodo relazionale legando ogni appello o esame programmato a uno specifico insegnamento già presente nel database locale. La sua architettura evidenzia scelte progettuali mirate alla stabilità del sistema:

- **Integrità Referenziale Asincrona (uselsFocused):** tramite l'hook **uselsFocused** di React Navigation, il componente intercetta il focus visivo della schermata ed esegue una chiamata asincrona (**getCorsi**) verso lo storage locale. Se la lista dei corsi è vuota, il modulo disabilita in modo sicuro il pulsante di salvataggio e mostra un testo di errore bloccante, impedendo la creazione di esami "orfani" privi di un legame relazionale. In modalità di modifica, ripristina l'associazione originaria mappando l'ID del corso (**corso_id**).
- **Selettore Dinamico Customizzato:** per evitare i bug di rendering del componente Picker (componente che serve a creare un menù a tendina o un menù di selezione a cascata) su sistemi operativi differenti, è stata implementata una **lista di selezione personalizzata** basata sul mapping di elementi **TouchableOpacity**. Il modulo evidenzia la scelta applicando dinamicamente uno stile condizionale (**opzioneCorsoSelezionata**) unito a un indicatore circolare personalizzato; in questo modo il sistema isola l'ID univoco del corso per scopi relazionali, lasciando a schermo solo il nome testuale per l'utente.
- **Controllo dei Vincoli Didattici:** prima di invocare le funzioni di persistenza dello storage (**salvaNuovoEsame**), il componente effettua una conversione (parsing) delle date in oggetti di tipo Date. Questo permette di confrontare la data dell'appello d'esame con la data di inizio del corso associato, annullando il salvataggio nel caso in cui un esame venga inserito erroneamente in una data antecedente all'inizio delle lezioni del corso stesso.

5.3 Gestione dello stato

La gestione dello stato adotta un approccio reattivo locale guidato dal ciclo di vita dei componenti e sincronizzato tramite due hook di sistema : **useEffect**, descritto precedentemente, e **useState**. L'hook **useState(d)** accetta un valore di default, detto **d**, e permette di:

1. associare al componente una **variabile di stato** (variable state) che serve a mantenere i dati persistenti tra i vari rendering; il valore di default della variabile di stato è **d**;

2. associare al componente una **state setter function**, ovvero una funzione che permette di aggiornare la variabile di stato e invocare il rendering del componente

L'applicazione rispetta rigorosamente il principio di immutabilità dello stato; tutte le operazioni di manipolazione dei dati non modificano mai direttamente gli array di origine, ma si avvalgono di metodi funzionali per generare proiezioni e copie pulite:

- Aggiunta di un elemento: questa operazione è implementata tramite l'operatore spread (**setItems([...items, newItem])**), che alloca un nuovo spazio in memoria unendo i vecchi record al nuovo oggetto tipizzato;
- Aggiornamento di stato : l'aggiornamento dello stato sfrutta il metodo **.map()** per scorrere l'array; se l'ID corrisponde al task selezionato, viene restituita una copia modificata dell'oggetto (**{...item, isCompleted: !item.isCompleted}**), altrimenti viene restituito l'elemento originario inalterato;
- Cancellazione fisica (**confirmDeleteTask**): questa operazione sfrutta il metodo **.filter()** per generare reattivamente un nuovo array privo dell'elemento il cui ID coincide con il valore della variabile temporanea **itemToDelete**.

Oltre alla reattività interna delle singole viste, l'applicazione risolve la sincronizzazione dello stato tra schermate differenti attraverso una strategia ibrida. Quando l'utente compila un form di inserimento separato (come **AddCorsoScreen.tsx** o **AddEsameScreen.tsx**), i dati non mutano direttamente lo stato locale della schermata precedente, ma vengono memorizzati nel database locale tramite le funzioni asincrone presenti nel file **storage.js**.

Il successivo aggiornamento delle interfacce principali (**Dashboard, Carriera, Planner**) viene garantito dall'azione combinata dell'hook **useIsFocused**, che intercetta il ritorno dell'utente sulla pagina e forza una rilettura immediata della memoria, aggiornando le rispettive state setter function e ricaricando l'interfaccia con le informazioni più recenti.

5.4 Gestione della navigazione

L'architettura della navigazione è implementata all'interno del file **AppNavigator.tsx** combinando due paradigmi integrati forniti dall'ecosistema React Navigation:

1. **Navigazione Orizzontale Primaria mediante la funzione **createBottomTabNavigator****: la funzione in questione è uno strumento fornito dalla libreria **ReactNavigation (@react-navigation/native-stack)** che serve a creare la barra di navigazione a schede posizionata in basso sullo schermo, chiamata **Bottom Tab Bar**; in tal modo, la funzione coordina il passaggio fluido tra le quattro macro-aree operative dell'app (**Dashboard, Planner, Academic, Timer**). La barra dei tab personalizzata occulta le etichette testuali (**tabBarShowLabel: false**) per massimizzare lo spazio visivo e utilizza un selettore condizionale per visualizzare le icone vettoriali corrette basandosi sullo stato di focus della rotta (focused);
2. **Navigazione Verticale di Dettaglio mediante **createNativeStackNavigator****: la funzione in questione restituisce un oggetto con due proprietà, **Screen** e **Navigator**, che rappresentano due componenti React usati per configurare il navigatore; il **Navigator** è il componente che implementa la navigazione tra schermate dell'app, mentre lo **Screen** è il componente che rappresenta la schermata effettiva per mostrare il contenuto. In particolare

`createNativeStackNavigator` opera come contenitore radice a pila (Stack); sovrappone cioè gerarchicamente alla barra dei tab le schermate di input (**AddCorso**, **AddEsame**, **AddScelta**) e di lettura analitica dei corsi (**CourseDetail**), garantendo transizioni fluide e mettendo a disposizione i comandi nativi di ritorno alla vista precedente (`navigation.goBack()`).

Nello sviluppo mobile, React Navigation mantiene le schede della TabBar caricate stabilmente in background per ottimizzare la velocità di risposta, impedendo il riavvio del ciclo di vita del componente (e quindi l'aggiornamento automatico dei dati) quando si cambia scheda. Per risolvere questa problematica e garantire la massima sincronizzazione visiva, le schermate **DashboardScreen.js**, **PlanningScreen.tsx** e **AcademicScreen.js** integrano l'hook nativo `useIsFocused`: questo hook restituisce un valore booleano (true o false) che indica se la schermata attuale è in quel momento visibile sullo schermo dell'utente oppure no. Legando l'esecuzione dei rispettivi hook `useEffect` alla variazione della variabile booleana `isFocused`, l'applicazione intercetta l'esatto istante in cui l'utente torna a visualizzare la pagina, forzando un'interrogazione asincrona immediata dei dati da `AsyncStorage`. Questo garantisce che le informazioni, i contatori e i grafici risultino costantemente aggiornati e coerenti tra le diverse schede.

5.5 Gestione dei dati persistenti

La persistenza offline dei dati è interamente affidata alla libreria **@react-native-async-storage/async-storage**, che salva i dati localmente sul dispositivo in modo asincrono, organizzandoli come un archivio di tipo chiave-valore.

Per garantire l'atomicità delle transazioni ed evitare colli di bottiglia, le chiamate di I/O sul file system sono centralizzate nel modulo di servizio **storage.js** attraverso funzioni asincrone e moduli di isolamento (`getAttività`, `getCorsi`, `salvaNuovaAttività`, ecc).

L'infrastruttura di storage si poggia sui due metodi fondamentali della libreria:

- **`AsyncStorage.getItem(key)` (Fase di Lettura)**: questo metodo viene attivato all'apertura delle pagine per caricare i dati della carriera e le attività/sessioni inserite nell'agenda. Poiché questo metodo legge la memoria del dispositivo e restituisce unicamente del testo semplice, il risultato viene intercettato da un costrutto `await`, che ferma l'esecuzione della specifica funzione finché il modulo di storage non ha recuperato i dati, e immediatamente deserializzato tramite `JSON.parse()` per riconvertire la stringa JSON in un array di oggetti JavaScript tipizzati e pronti all'uso;
- **`AsyncStorage.setItem(key, value)` (Fase di Scrittura)**: questo metodo viene eseguito automaticamente ad ogni inserimento, modifica o cancellazione. Prima di inoltrare il dato al metodo, l'array di oggetti (ad esempio l'elenco aggiornato degli items) viene serializzato in stringa testuale continua tramite `JSON.stringify()` , l'unico formato compatibile con i vincoli architetturali di memorizzazione della libreria

Nelle schermate operative, il ciclo di persistenza reattiva è protetto da un apposito "semaforo" logico per prevenire la corruzione dei dati: all'avvio di un componente (ad esempio `PlanningScreen.tsx`) lo stato iniziale nasce inevitabilmente vuoto (`[]`). Di conseguenza, il salvataggio automatico viene bloccato da un controllo iniziale (`if (!isLoading) return;`). Questa barriera impedisce al sistema di

sovrascrivere la memoria del dispositivo con una lista vuota prima che il caricamento dei dati reali sia giunto a termine.

5.6 Eventuali parti particolarmente significative e complesse

Nel codice, sono presenti alcune sezioni che meritano un'attenzione particolare:

- **Pipeline Funzionale di Filtraggio in Cascata con Ottimizzazione ID (PlanningScreen.tsx)**

Una sezione degna di nota è collocata all'interno della proprietà **data** della **FlatList** in **PlanningScreen.tsx**: per soddisfare i requisiti di ricerca e filtro senza sovraccaricare la memoria del dispositivo con stati duplicati o variabili ridondanti, è stata sviluppata una **pipeline di metodi .filter()** nativi concatenati direttamente in linea.

Inoltre, al fine di garantire l'isolamento della logica e la massima stabilità in contesti di sviluppo collaborativo, l'intero algoritmo di ricerca e filtro è stato standardizzato sugli ID univoci dei corsi (**corso.id**), risolvendo ogni conflitto con le chiavi esterne e isolando il cambiamento all'interno della vista, mantenendo la visualizzazione del nome testuale del corso confinata alla sola interfaccia utente;

- **Flusso d'Integrazione e Integrità Referenziale nella Verbalizzazione (AcademicScreen.js)**

Un ulteriore elemento di complessità implementativa è rappresentato dalla routine di **Verbalizzazione dell'Esame** situata in **AcademicScreen.js**. Quando lo studente tocca il pulsante di spunta di un appello programmato, l'app attiva un overlay modale che avvia un'operazione di aggiornamento incrociato e simultaneo sui dati persistenti:

- Il componente esegue una ricerca inversa nell'array dei corsi sfruttando la chiave esterna **esame.corso_id** per individuare il nome dell'insegnamento associato, mostrandolo come feedback di sicurezza nel modale.
- Qualora l'esame venga contrassegnato come **SUPERATO**, l'utente digita il voto ottenuto (convalidato e limitato nel range 18-30). Al clic sul pulsante di conferma, viene invocata la funzione di storage **verbalizzaEsitoEsame**.
- La routine interviene simultaneamente su due chiavi distinte del database locale: aggiorna lo stato dell'entità esame in 'superato' inserendo il voto in **voto_risultato** e, contestualmente, localizza l'entità corso padre, ne cambia lo stato in 'completato' e inserisce il voto nel rispettivo campo **voto_ottenuto**. Questo automatismo garantisce la perfetta integrità referenziale dei dati e aggiorna istantaneamente i tracciati dei grafici del cruscotto al successivo cambio di tab.

- **Meccanismo del Pulsante Centrale Fluttuante e Navigazione di Immissione Rapida (AppNavigator.tsx)**

Una menzione speciale va riservata all'architettura sviluppata all'interno del file **AppNavigator.tsx** per ottimizzare l'esperienza di acquisizione dei record (Corsi, Esami, Task). Per rendere l'inserimento dei dati più rapido e immediato, è stato inserito un pulsante "+" centrale fluttuante (**CustomTabBarButton**) direttamente nella barra dei tab, che si

sovrappone geometricamente all'interfaccia elevandosi di 20 pixel (**top: -20**) e offrendo un forte contrasto visivo e un'ombreggiatura marcata (**elevation: 5**).

Al tocco del pulsante, anziché caricare una schermata, l'app solleva un **Modal** trasparente che funge da menu a comparsa. Per garantire un passaggio fluido tra le schermate ed evitare rallentamenti o scatti grafici, viene impiegata la funzione **navigateAndClose**: questa funzione disattiva prima lo stato di visibilità del modale e solo successivamente, sfruttando un'attesa asincrona controllata di 100ms mediante il metodo `setTimeout`, ordina al navigatore a pila di eseguire il push verso la schermata di inserimento richiesta (`AddCorso`, `AddEsame` o `AddScelta`). In questo modo, l'interfaccia esegue le animazioni in tempi separati ed evita i conflitti grafici sul thread principale dell'applicazione.